AsanaAI: System and Model Architecture

Technical design document

Anamitra Sarkar Debjit Deb Barman Adrija Ray Mondal Tanima Samanta
Supervisor: Mr. Sujit Chakraborty, Assistant Professor, Department of CSE (AI/ML)
RCC Institute of Information Technology, Kolkata, West Bengal, India
Final-year project P05

Scope. This document describes how AsanaAI is built: its components, the path of one camera frame through the system, the three trained models and the rule engine, and how everything is deployed. It contains no accuracy results; those are in the research paper and the project report. Live app: https://yoga-posture-correction-system.vercel.app. API: https://arko007-yoga-pose.hf.space.

Contents

1 System Overview **2**

2 Design Principles **2**

3 Client Architecture (in the Browser) **4**
3.1 The path of one frame . . . 4
3.2 Pose engine and its fallback ladder . . . 4
3.3 From landmarks to the 15 joint angles . . . 4
3.4 Visibility handling . . . 5
3.5 Skeleton buffer for the sequence model . . . 5
3.6 Request cadence, timeouts and offline mode . . . 5
3.7 Interface . . . 5

4 Server Architecture **6**
4.1 Hosting and process model . . . 6
4.2 Endpoints . . . 6
4.3 Handling one frame . . . 7
4.4 Handling one window . . . 7
4.5 Configuration and rollback . . . 7

5 The Models **7**
5.1 What goes into each model . . . 7
5.2 The 3-head residual MLP . . . 7
5.3 The cascade: one model names, another vetoes . . . 8
5.4 The rule engine and per-joint corrections . . . 9
5.5 The ST-GCN sequence model . . . 9
5.6 How the outputs reach the screen . . . 11

6 Calibration, Visibility and Coaching **11**
6.1 Personal range-of-motion calibration . . . 11
6.2 Visibility-aware scoring . . . 11
6.3 Safe, multilingual coaching . . . 11

7 Deployment Topology and Operations **12**

8 How the Models Are Produced **12**

9 Repository Map **13**

1 System Overview

AsanaAI is a yoga coach that watches a person through a camera, names the pose they are in, scores their form, and tells them in English, Hindi or Bengali what to adjust, on screen and aloud. It runs as a web app (a progressive web app that can be installed on a phone) backed by a small CPU server. The central design choice is that the camera image never leaves the device: the browser turns each frame into 33 body landmarks, and only numbers derived from them are sent to the server. Figure 1 (on its own page after Section 2) shows the whole data flow, and Table 1 lists the components.

Table 1: Components, where they run and what they do.

Component	Runs on	Role
Web app	Browser (Next.js 14, React 18, next-pwa service worker)	Camera, pose engine, joint angles, skeleton buffer, interface, speech, local storage of the user's calibration profile
Pose engine	Browser (GPU, or CPU in a Web Worker, or WASM)	Image $\rightarrow$ 33 landmarks (x, y, z, v)
On-device coach (basic mode)	Browser (TypeScript)	Port of the rule engine and coaching templates; used when the server is unreachable
Android app	Capacitor 8 wrapper (plus an Expo companion app)	Packages the same web app for Android
API server	Hugging Face Space (FastAPI, PyTorch, CPU)	Frame analysis, sequence analysis, coaching text
Pose MLPs (namer, gate)	Server	Name the pose; veto "not one of my poses"; form score
ST-GCN	Server	Second opinion from 60 frames of skeleton motion
Rule engine	Server (and browser)	Per-joint deviations from reference angle bands
Coaching service	Server (calls an external language model)	Template $\rightarrow$ optional paraphrase $\rightarrow$ safety screen
Model repository	Hugging Face Hub	Stores weights and class encoders; the server downloads them at start-up

2 Design Principles

1. **Privacy by construction.** The request carries 15 joint angles, or 60 frames of landmark coordinates, never pixels. There is no code path that uploads an image.
2. **Small models on a CPU.** The two pose MLPs have 0.36 M parameters each and the ST-GCN 0.89 M; all three weight files together are 6.6 MB. The server needs no GPU.
3. **Abstain rather than guess.** A second model can veto a pose name ("not one of my poses"); joints the camera cannot see are neither scored nor coached; the sequence model only speaks when it is confident.
4. **Graceful degradation.** Every layer has a fallback: three pose engines, an on-device coach when the server is asleep, the original decision path if the gate model fails to load, and a reviewed template if the language model is unavailable or produces an unsafe sentence.
5. **Roles for each component.** Neural networks name the pose and judge form; the rule engine supplies per-joint corrections because they are defined by reference angle bands; the language model may only rewrite a reviewed sentence.

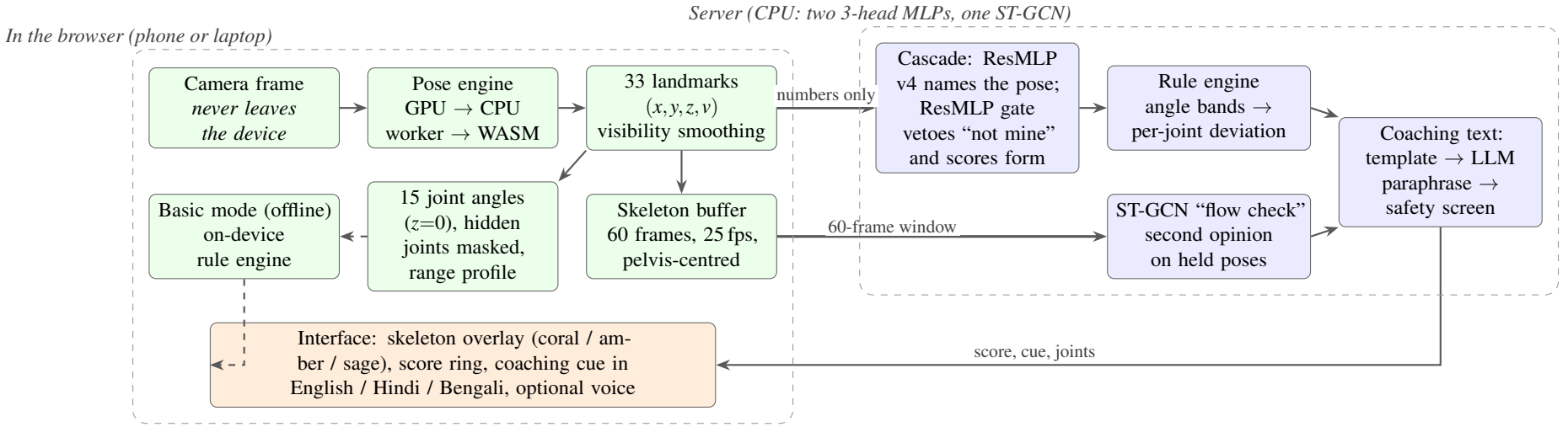


Figure 1: End-to-end architecture. Everything on the left runs in the browser: the camera frame is converted to 33 landmarks by a pose engine chosen automatically, hidden joints are masked, and only numbers are sent. The server holds three small neural networks and a rule engine. Dashed arrows show the offline path, in which an on-device port of the rule engine keeps a basic mode working.

3 Client Architecture (in the Browser)

3.1 The path of one frame

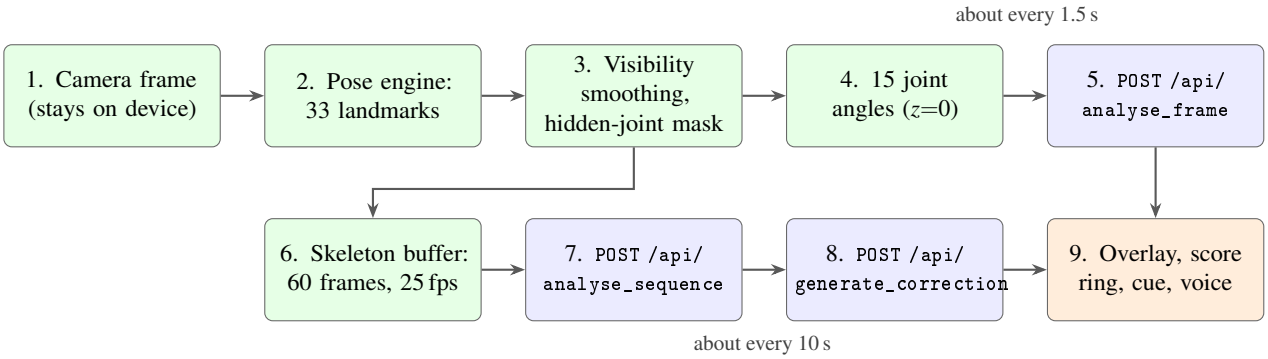


Figure 2: The client pipeline. Steps 1–4, 6 and 9 run on the device (green and orange); 5, 7 and 8 are the three network calls to the server (blue).

3.2 Pose engine and its fallback ladder

The pose engine returns 33 MediaPipe landmarks per frame, each as (x, y, z, v) where v is the estimator's visibility. Different phones can break different engines (a GPU driver can crash MediaPipe), so the app picks one automatically (Figure 3). The decision is remembered per browser version and is invisible to the user.

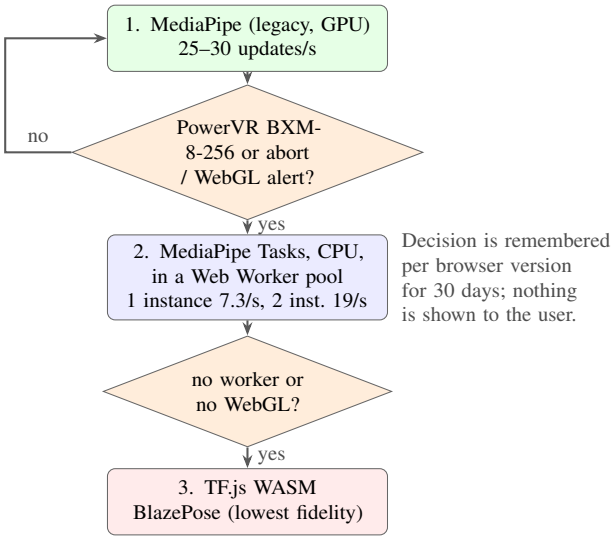


Figure 3: Pose-engine fallback ladder: MediaPipe on the GPU; MediaPipe Tasks on the CPU in a Web Worker pool (a phone with at least six cores runs two instances and alternates frames); TensorFlow.js with a WebAssembly BlazePose as the last resort.

3.3 From landmarks to the 15 joint angles

Every angle is the planar angle at a *vertex* landmark between two other landmarks, computed as $\theta = \arccos\left(\frac{(\mathbf{a}-\mathbf{b})\cdot(\mathbf{c}-\mathbf{b})}{\|\mathbf{a}-\mathbf{b}\|\,\|\mathbf{c}-\mathbf{b}\|}\right)$ in degrees, with the z coordinate set to zero. The angles are relative, so they do not change if the whole body rotates or moves in the frame. The same 15 numbers, in the same fixed order, are computed in TypeScript in the browser and in Python on the server (the Python function mirrors the TypeScript one).

Why $z = 0$. MediaPipe's depth is a monocular relative estimate that is only reliable at a fixed camera distance and framing. The angle models therefore use the planar angles, which the browser can compute cheaply and identically on any camera. The ST-GCN, in contrast, receives the full (x, y, z) coordinates (Section 5.5).

Table 2: The 15 features, in the order the models expect. *L*/*R*: the subject's left/right. Landmark numbers are MediaPipe indices.

#	Name	Angle at the vertex, between	Landmarks (a, vertex, c)
1,2	`elbow_l/r`	shoulder – **elbow** – wrist	11,13,15 / 12,14,16
3,4	`shoulder_l/r`	hip – **shoulder** – elbow	23,11,13 / 24,12,14
5,6	`hip_l/r`	shoulder – **hip** – knee	11,23,25 / 12,24,26
7,8	`knee_l/r`	hip – **knee** – ankle	23,25,27 / 24,26,28
9,10	`ankle_l/r`	knee – **ankle** – heel	25,27,29 / 26,28,30
11,12	`trunk_l/r`	shoulder – **hip** – opposite hip	11,23,24 / 12,24,23
13	`neck`	nose – **mid-shoulder** – mid-hip	0, mid(11,12), mid(23,24)
14,15	`hip_abduct_l/r`	opposite hip – **hip** – knee	24,23,25 / 23,24,26

3.4 Visibility handling

The estimator's per-landmark visibility is smoothed (exponential average) with hysteresis: a landmark becomes hidden when its smoothed visibility falls below 0.4 and visible again only above 0.6, so a joint on the edge does not flicker. Each angle feature depends on a fixed set of landmarks (the three landmarks of Table 2; the neck angle depends on the nose, both shoulders and both hips), so a hidden landmark invalidates exactly the features built on it; for example, a hidden left elbow (landmark 13) invalidates `elbow_l` and `shoulder_l`. Hidden joints are not scored, not coached, and are drawn faint; the interface states which joints it is not checking.

3.5 Skeleton buffer for the sequence model

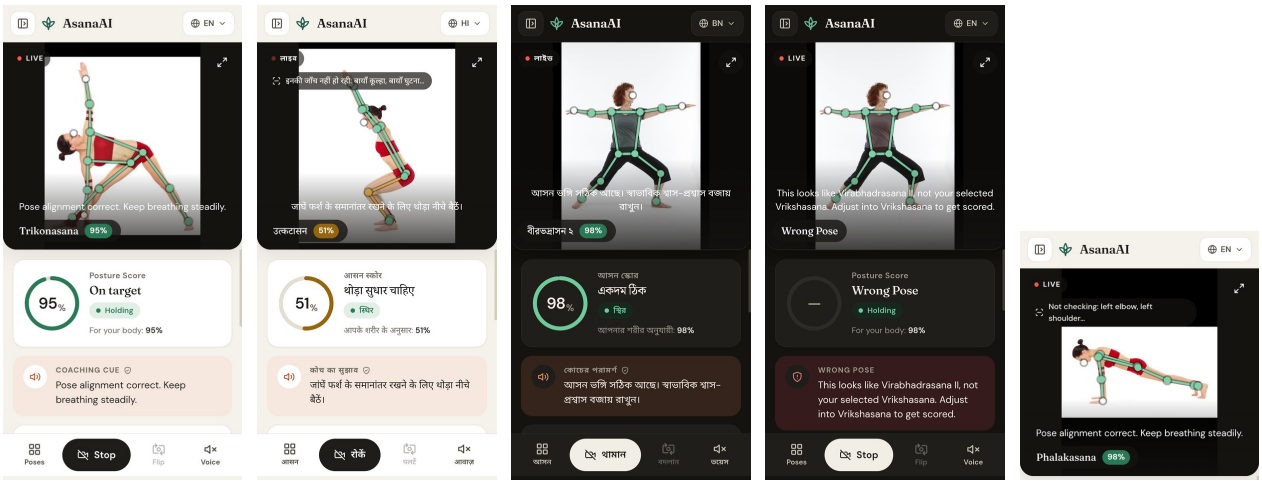
The ST-GCN needs 60 consecutive frames at the rate it was trained on (about 25 fps, a 2.4 s window). A time-stamped buffer is filled from *every* camera result (not from the slower server loop), keeps the last 4 s, and returns a window resampled to exactly 60 frames at 25 fps by linear interpolation. A landmark with visibility below 0.5 keeps its last good position; if the camera stalls for more than 250 ms inside the window the window is invalid and the app shows "static check" instead of a sequence answer.

3.6 Request cadence, timeouts and offline mode

The pose and score refresh about every 1.5 s; coaching text, speech and the sequence call run on a slower cycle of about 10 s. Calls carry a hard timeout so that a sleeping server never hangs the app. After two consecutive failures the app switches to *basic mode*: a TypeScript port of the rule engine and the coaching templates that answers instantly on the device (rules only: no MLP, no gate, no ST-GCN, no language model; the interface says so). It probes the server again every 15 s and returns to full mode when it answers. The port was verified against the Python original on generated cases.

3.7 Interface

The interface draws the skeleton over the camera view (sage: inside the reference band, amber: more than 7° outside, coral: more than 15° outside), a posture-score ring, a coaching cue in the chosen language with optional spoken output, and the pose chosen by the server. Two modes exist: *free mode* names whatever the user does; *guided mode* lets the user select one of 8 target poses (Warrior II, Tree, Triangle, Chair, Downward Dog, Plank, Seated Easy, Corpse), scores the body against that pose, and shows a wrong-pose card if the camera sees another pose.



(a) Free mode **(b)** Hindi cue, amber **(c)** Bengali, dark theme **(d)** Guided: wrong pose **(e)** Hidden joints not judged

Figure 4: The deployed interface on a POCO M7 Pro, with a credited Wikimedia photo standing in for the camera. (a) Trikonasana with the posture-score ring and the "for your body" personal score. (b) Utkatasana: the cue in Hindi, a score in amber, and a chip naming the joints not checked. (c) Virabhadrasana II, Bengali, dark theme. (d) Guided mode with Vrikshasana selected while a Warrior II photo is shown: the score is withheld and the app names the pose it sees. (e) Plank in guided mode: hidden limbs are drawn faint and are neither scored nor coached. Photos: Kennguru (a, b, e; CC BY 3.0), lululemon athletica (c, d; CC BY 2.0), via Wikimedia Commons.

4 Server Architecture

4.1 Hosting and process model

The API is a FastAPI application (`backend/app/main.py`) served from a Hugging Face Space. The Space uses the Gradio SDK: its entry point `backend/app.py` mounts a small demo page and imports the same FastAPI application, so every `/api/*` route is available on the Space URL. At start-up the server downloads the model files from the Hugging Face model repository and caches them; PyTorch is limited to a fixed number of CPU threads. Cross-origin access is restricted to the production web origin and local development origins. The endpoints are stateless: anything that needs history (for example motion) is computed by the client and sent in the request, so the Space can run several workers.

4.2 Endpoints

Table 3: The API (all routes under `/api`, JSON).

Route	Request	Response
`POST /analyse_frame`	`angles` (15 floats, required); optional `calibration`, `motion`, `target_pose`, `world_angles`, `orientation`	`pose_id`, `correctness_score`, `deviations` (15 joints, degrees), `motion_state`, top-3 `candidates`, `cascade` info, optional `personal_correctness_score`, `calibrated_deviations`, `guided`
`POST /analyse_sequence`	`coordinates`: exactly 60 frames × 99 numbers	`sequence_pose`, `confidence`, `requires_static_fallback`, `sequence_kind`
`POST /generate_correction`	`pose_id`, `deviations`, `language`, `attempt` (how many times this cue was already delivered without the joint improving)	`correction_text` (template or screened paraphrase), `is_safe`, `target_joint`
`POST /occlusion_recovery`	`mp_landmarks` (33×4), optional `cliff_landmarks`	Optional hook for mesh-based recovery of hidden joints; not used by the deployed app
`GET /health`	–	Service status

4.3 Handling one frame

`analyse_frame` validates that exactly 15 angles arrived, runs the namer and the gate on the same input tensor (a few milliseconds on a CPU), and passes both outputs to the cascade decision (Section 5.3). In the cascade the form score is the gate's correctness logit passed through a sigmoid, and the per-joint deviations are the distances outside the pose's angle bands; the deviation head's output (multiplied by 180 and clipped to $[0°, 180°]$) is used only on the fallback path. If the request carried a calibration profile, a second, personalised score is computed (Section 6.1); if it carried a target pose, a `guided` block scores the body against that pose. `motion_state` ("holding", "transitioning", "unrecognized") comes from the angular speed the client measured over the last 1.5 s: you do not correct someone mid-movement, but you do tell them when a held posture is not recognised.

4.4 Handling one window

`analyse_sequence` requires exactly 60×99 numbers. The window is translated so the pelvis (midpoint of landmarks 23 and 24) is the origin, divided by the hip width, and passed to the ST-GCN; the softmax top class and its confidence are returned. The answer is flagged `requires_static_fallback` when the class is `transition/unknown` or the confidence is below 0.70 (0.55 for child's pose); the app then shows "static check" and relies on the per-frame result alone.

4.5 Configuration and rollback

Table 4: Server switches (environment variables, set as Space variables). Defaults equal the production setting.

Variable	Default	Effect
`ENABLE_POSE_CASCADE`	`1`	`0` reverts to the original single-model path without a redeploy
`GATE_MODEL_FILE`, `GATE_ENCODER_FILE`	`mlp_3head_gate_v1.pth` and its encoder	Gate checkpoint; if it cannot be loaded the endpoint answers with the original path and reports the reason in `cascade.reason`
`STGCN_MODEL_FILE`, `STGCN_ENCODER_FILE`	`stgcn_target_v1.pth` and its encoder	Sequence checkpoint; older checkpoints stay in the repository and can be selected here
`GATE_REPO`	`Arko007/yoga-posture-models`	Where the gate is downloaded from (the namer and the sequence model use the `HF_REPO` setting in `config.py`, the same repository)

5 The Models

5.1 What goes into each model

- The two **pose MLPs** (namer and gate) read the *15 planar joint angles* of one frame (Table 2).
- The **ST-GCN** reads a *window of 60 frames* $\times$ *33 joints* $\times$ (x, y, z), pelvis-centred and divided by hip width, so it is independent of where the person stands and how far from the camera.

This division is deliberate: the MLPs are a single-frame form check from view-independent angles; the ST-GCN adds the skeleton's graph structure, the temporal flow and depth.

5.2 The 3-head residual MLP

Both the namer and the gate are the same architecture, `Yoga3HeadMLP` (`backend/app/models/mlp.py`). A shared trunk feeds three heads, so one forward pass answers "which pose, how well, which joints".

- **Pose head**: the class scores. The class list comes from the model's encoder file; classes named `imperfect_<pose>` are summed onto their base pose at serving time.
- **Form head**: one logit, trained as a "correct versus deliberately imperfect or non-pose frame" probability; a sigmoid turns it into the form score.

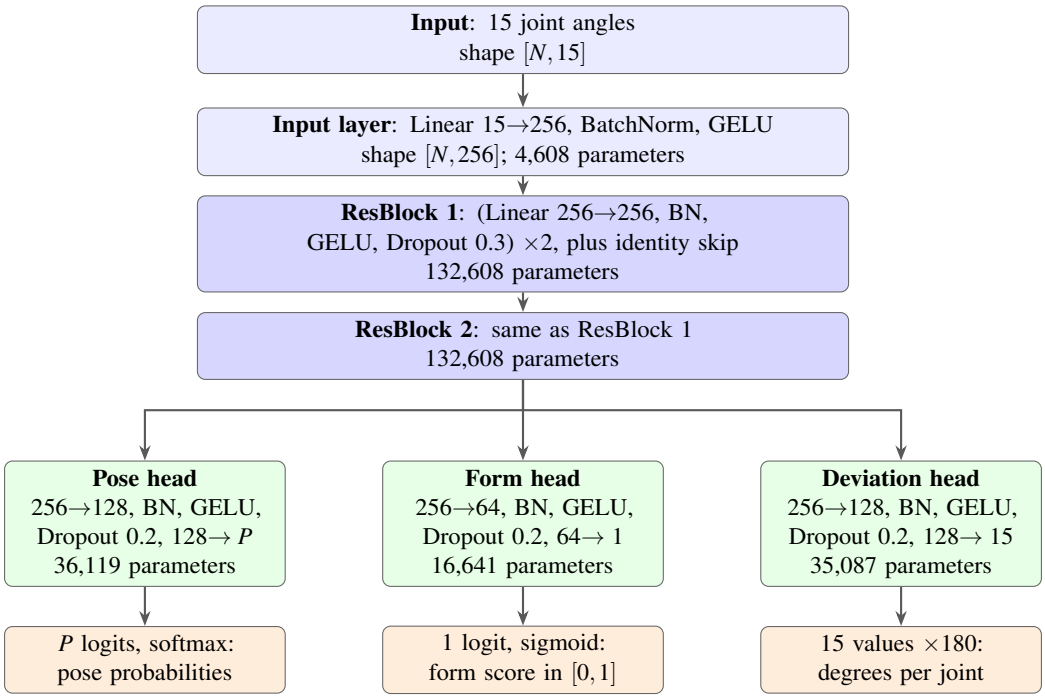


Figure 5: The 3-head residual MLP. $P = 23$ pose classes in both deployed models; 357,671 parameters in total (counted from the model definition). Shapes are for a batch of N frames.

- **Deviation head**: how far each of the 15 joints is from correct, in units of $1/180$.

Residual connections let the 256-wide trunk train stably; batch normalisation and dropout regularise a model that is small enough to run in a few milliseconds.

5.3 The cascade: one model names, another vetoes

Two checkpoints of this architecture are used together. The **namer** is the pose model whose top class is the answer. The **gate** was trained with an explicit "none of our poses" class (`transition/unknown`); it answers the question "is this one of my poses at all?" and supplies the form score. The rule engine does not override either model on the pose *name*; it only supplies per-joint deviations.

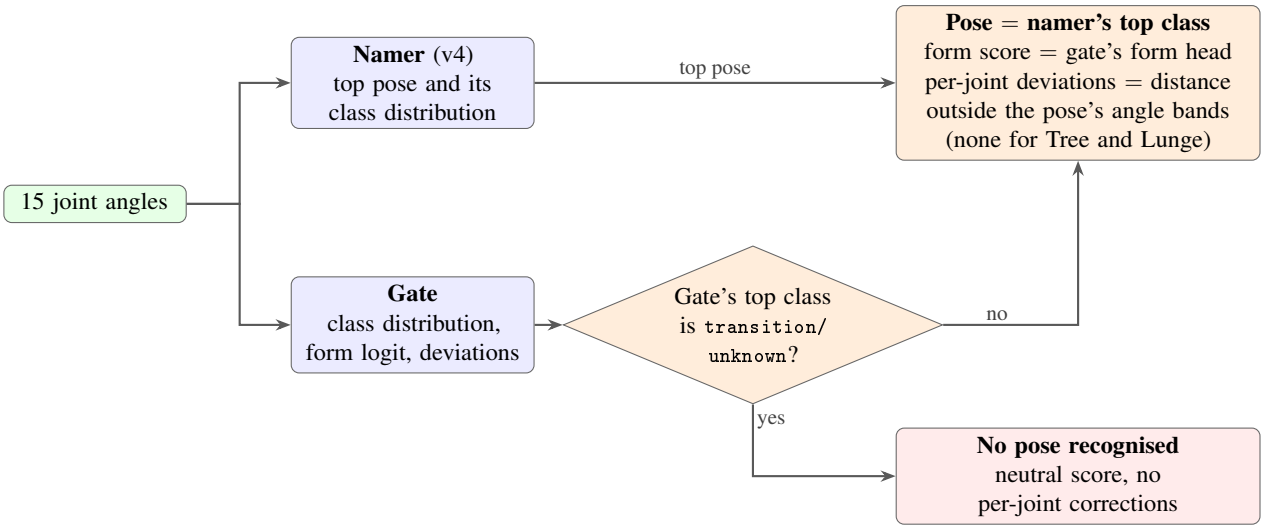


Figure 6: The per-frame decision. The namer and the gate see the same 15 angles. The answer is the namer's pose unless the gate says the frame is not one of the app's poses.

In code (`backend/app/services/cascade.py`, pure logic without PyTorch, so it is unit-tested directly):

```
gate_pose = argmax(collapse(gate_distribution))
gated     = (gate_pose == "transition/unknown")
```

```
pose      = "transition/unknown" if gated else sanitize(namer_pose)
if pose == "transition/unknown":
    return no_pose_result                       # neutral score, zero deviations
form      = sigmoid(gate_correctness_logit)     # form score from the gate
if pose has angle bands:   deviations = distance_outside_bands(pose, angles)
else:                      deviations = zeros  # Tree and Lunge: no per-joint claims
```

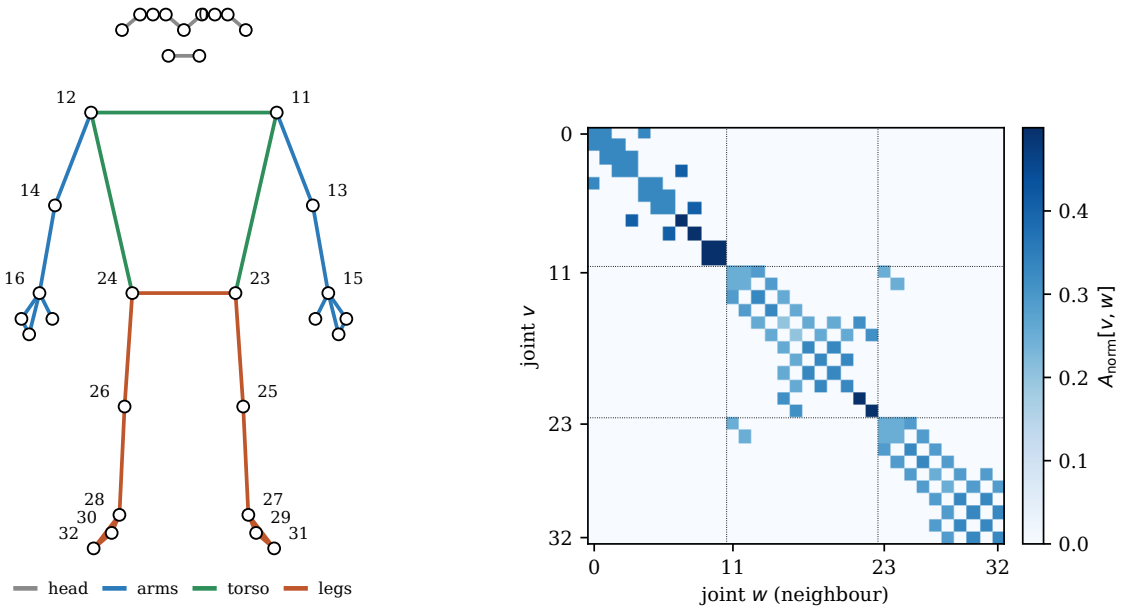
The response also reports the namer's top three candidates and the gate's view (`cascade`: `active`, `gated`, `gate_pose`, `gate_other_prob`, and the sources of the score and deviations), so a client can show ambiguity honestly. If the gate cannot be loaded, or the switch is off, the endpoint falls back to the original path (the namer plus the rule engine's vote) and says so in the response.

5.4 The rule engine and per-joint corrections

Each pose has a table of reference angle bands, for example for Downward Dog: knee in $[110°, 180°]$, shoulder in $[95°, 180°]$, hip in $[20°, 140°]$. The *deviation* of a joint is the distance in degrees outside its band (zero if inside); the rule score is $1 - \bar{d}/45°$ clipped to $[0, 1]$. The vocabulary has 19 poses (mountain, upward salute, downward dog, plank, cobra, child, seated staff, seated easy, tree, warrior I, warrior II, lunge, standing forward fold, halfway lift, chair, table top, standing, triangle, corpse); 17 have bands. Tree and Lunge have no bands, so the system makes no per-joint claim for them and draws neutral limbs. Because the learned deviation head is the weakest of the three, joint-level corrections and limb colours come from the bands wherever they exist. The same table is implemented in Python (server) and TypeScript (basic mode).

5.5 The ST-GCN sequence model

Why a graph network. A skeleton is not a grid: its joints are connected in a fixed way. A spatial-temporal graph convolutional network (ST-GCN) treats the 33 MediaPipe landmarks as the nodes of a graph whose edges are the bones, and slides a temporal filter over 60 frames, so each joint's features are mixed with its anatomical neighbours and with its own recent past.



(a) The graph: 33 nodes, 35 bones

(b) A_{norm} (33×33, 103 non-zero entries)

Figure 7: The skeleton graph used by the ST-GCN (drawn from the model's own definition). (a) MediaPipe landmarks and the 35 skeletal edges (numbers are landmark indices; subject's left is on the right). (b) The symmetric-normalised adjacency $A_{\text{norm}} = D^{-1/2}(A+I)D^{-1/2}$: block structure shows the head, the arms and torso, and the legs; the diagonal is the self-loop.

One block. A block (Figure 8) applies (i) a 1×1 convolution that projects each joint's channels, (ii) a graph aggregation $\mathbf{Y}[c,v,t] = \sum_w A_{\text{norm}}[v,w]\,\mathbf{X}[c,w,t]$ implemented as `einsum('vw,ncwt->ncvt')`, (iii) GELU, (iv) a temporal convolution with a 9×1 kernel over the 60 frames for each joint independently, followed by batch normalisation and dropout, and (v) a residual connection, with a 1×1 convolution and batch normalisation on the skip path when the channel count changes.

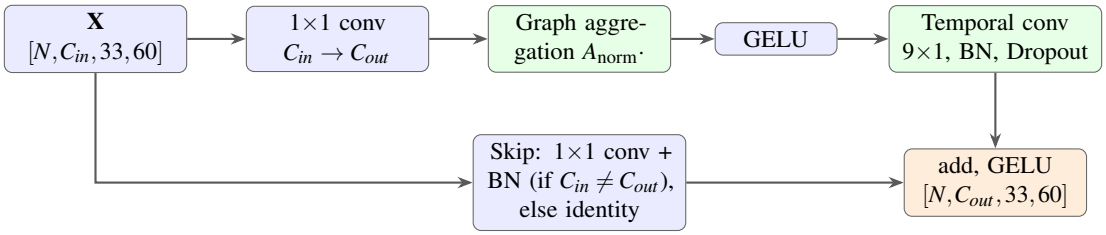


Figure 8: Inside one ST-GCN block. The graph step mixes each joint with its neighbours; the temporal step mixes each joint with its neighbours in time.

The whole network. Three blocks raise the channel count $3 \to 64 \to 128 \to 256$ (no temporal striding, so all 60 frames are kept), the features are averaged over all joints and frames, and a small head produces the class scores.

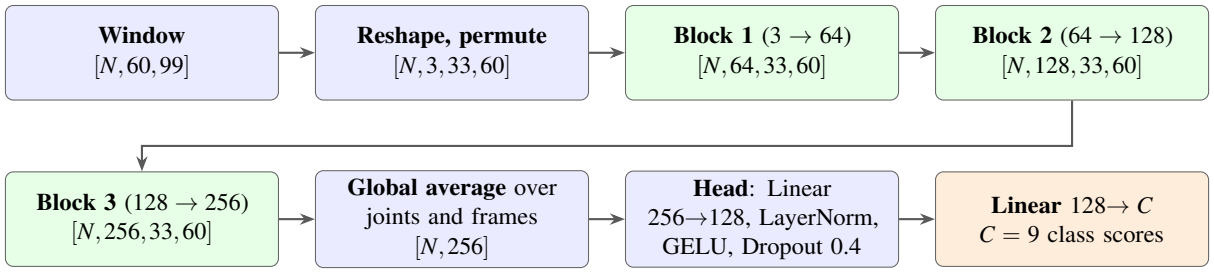


Figure 9: ST-GCN data flow with tensor shapes, read left to right and then continued on the second row. N is the batch size; the input has 60 frames, 33 joints and three coordinates per joint.

Table 5: ST-GCN parameter budget (counted from the model definition; $C = 9$).

Stage	Graph conv	Temporal conv	Skip	Total
Block 1 $(3 \to 64)$	256	37,056	384	37,696
Block 2 $(64 \to 128)$	8,320	147,840	8,576	164,736
Block 3 $(128 \to 256)$	33,024	590,592	33,536	657,152
Head $(256 \to 128 \to 9)$				34,313
Total				**893,897**

Output vocabulary and role. The deployed network (`stgcn_target_v1`) has 9 classes: child's pose, corpse, downward dog, plank, seated easy, tree, triangle, warrior II, and `transition/unknown`. It acts as a second opinion over the last 2.4 s (the "flow check") and is shown only when confident; it never overrides the per-frame pose. The Python class is called `YogaSequenceLSTM` for checkpoint compatibility only; it contains no recurrent layer.

5.6 How the outputs reach the screen

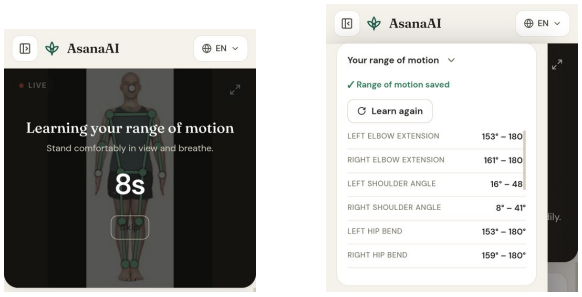
Table 6: Which component produces each thing the user sees.

On screen	Produced by
Pose name	Namer, unless the gate vetoes (then "not recognised"); in basic mode, the rule engine
Posture score (ring)	Gate's form head; "for your body" score from the personal calibration
Limb colours and per-joint cue	Rule-engine angle bands (none for Tree and Lunge); hidden joints excluded
Flow check / static check	ST-GCN if its confidence passes the threshold, otherwise "static check"
Coaching sentence	Template → optional language-model paraphrase → safety screen
Wrong-pose card (guided mode)	Detected pose differs from the chosen target pose

6 Calibration, Visibility and Coaching

6.1 Personal range-of-motion calibration

Bodies differ, so a joint inside its owner's comfortable range is "different", not "wrong". At camera start the app can run a 15-second calibration while the user stands comfortably in view. For each of the 15 joints the profile stores the observed minimum and maximum widened by 15° (clamped to $[0°, 180°]$) and the mean as the resting angle; a profile with fewer than 20 frames is discarded. The profile lives only in the browser's local storage and is sent as numbers with each request. On the server a joint inside its personal range has its deviation set to zero and a second, personalised score is returned; it is never lower than the universal score and equals it when nothing is forgiven.



(a) Countdown **(b)** Saved ranges

Figure 10: Calibration screens: (a) 15-second capture; (b) the learned per-joint ranges, stored on the device.

6.2 Visibility-aware scoring

Instead of guessing hidden joints, the system abstains: landmarks with low smoothed visibility are excluded from the score, from the per-joint cue and from the limb colouring, and the interface lists what is not being checked. (A server hook for mesh-based recovery of hidden joints exists in the code but is not used by the deployed app, because it would need the camera image, which never leaves the device.)

6.3 Safe, multilingual coaching

The language model is a 27-billion-parameter Qwen model reached through a hosted API (8 s timeout, temperature 0.2). The screen discards any paraphrase containing "push", "force", "hurt", "pain", "stretch more" or "further". This English list is applied to every language, because a model can slip an English word into a Hindi or Bengali answer; equivalent Hindi and Bengali word lists, reviewed by a native speaker of both languages, cover the other two languages. Cues escalate on a fixed ladder: the second delivery of a cue that produced no measured improvement adds how many degrees off the user is; from the third time the cue backs off (ease out of the shape and come back only as far as feels comfortable), because a cue that failed twice usually means the range is not available today.

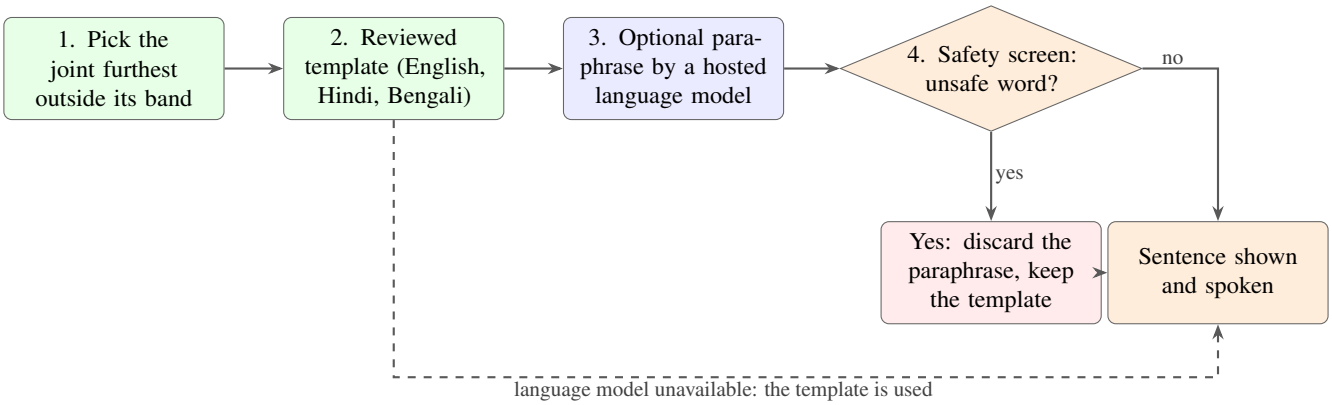


Figure 11: The coaching pipeline. The language model may only rewrite a reviewed sentence; the template is always the fallback.

7 Deployment Topology and Operations

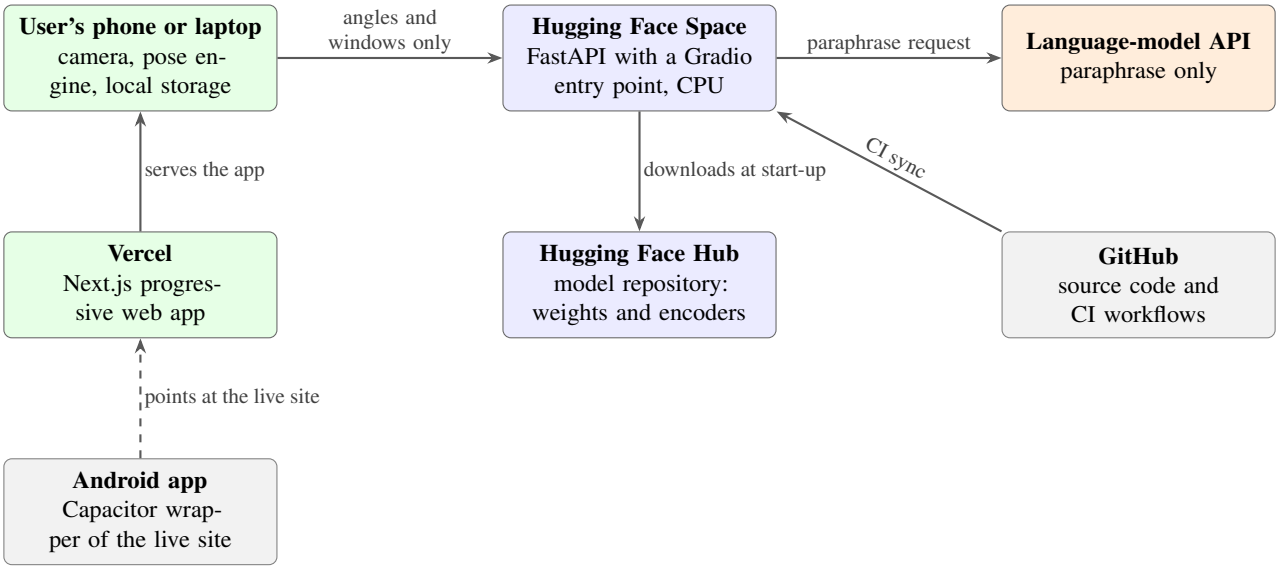


Figure 12: Deployment topology. The only data that go from the device to the server are joint angles and landmark windows; the camera image stays in the browser.

- **Web.** The Next.js app is hosted on Vercel and works as a progressive web app: the pose-engine files are cached after the first visit so it also starts offline.
- **API.** A GitHub Actions workflow pushes the backend to the Hugging Face Space whenever `backend/**`, `README.md` or the workflow itself changes. Changes to documents or training scripts do not redeploy it.
- **Android.** A workflow builds a Capacitor debug APK; it is configured in server mode, so it simply points at the live web app. A separate Expo/React Native companion app lives in `mobile/`.
- **Weights.** Checkpoints are never overwritten: a new model is added under a new file name, and the active one is selected by a variable, so rolling back is a configuration change.
- **Cost and footprint.** The server runs on a CPU Space; with all three networks loaded it needs about 324 MB of memory, and server compute is roughly 7 to 9 ms per frame request.

8 How the Models Are Produced

Each checkpoint is accompanied by an encoder file (`.npy`) holding the class names in output order. The training data are published as the Yoga-1M dataset (https://huggingface.co/datasets/Arko007/Yoga-1M) and the weights as https://huggingface.co/Arko007/yoga-posture-models; both have a card describing exactly which data trained which model.

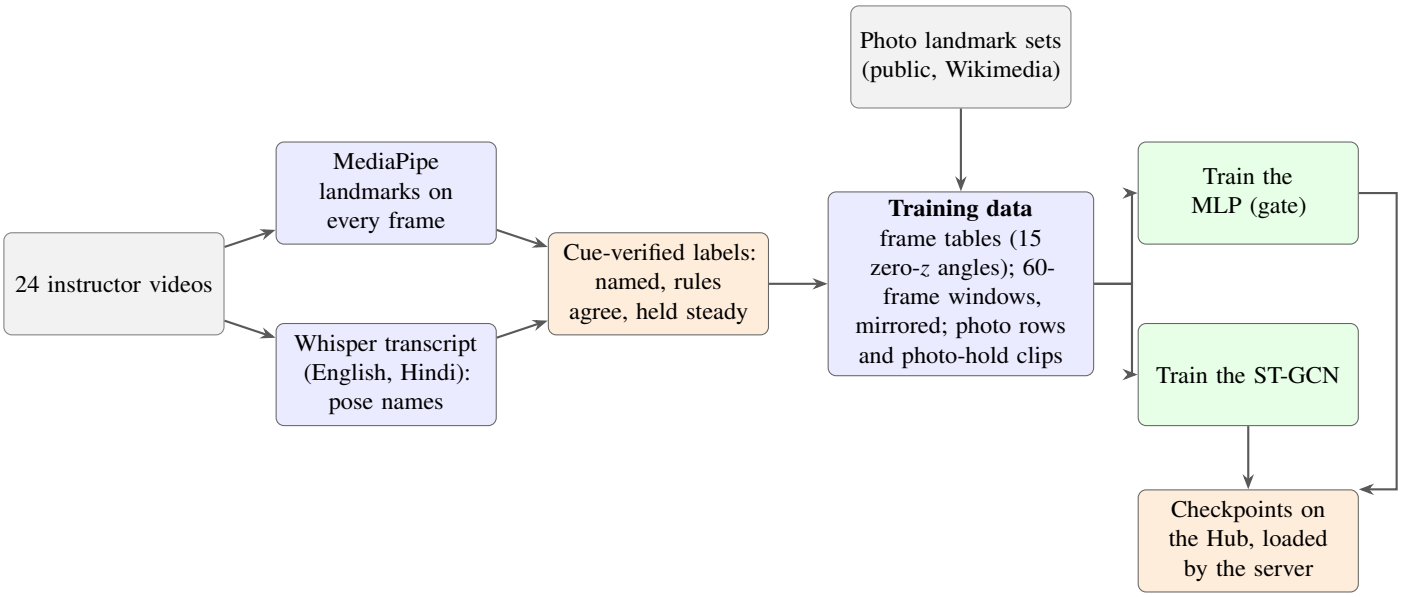


Figure 13: The model production pipeline. Video frames are labelled only where three independent sources agree; photos add variety that 24 videos cannot. The namer is an earlier checkpoint trained before the cue-verified labels existed (on a 12-video table plus photos).

Table 7: The three production checkpoints (repository `Arko007/yoga-posture-models`).

Role	Weights	Parameters	Size	Output
Namer	`mlp_3head_v4_photos_x2000.pth`	357,671	1.5 MB	P=23 pose scores, form, 15 deviations
Gate	`mlp_3head_gate_v1.pth`	357,671	1.5 MB	P=23 pose scores (with "none of mine"), form, 15 deviations
Sequence model	`stgcn_target_v1.pth`	893,897	3.6 MB	9 classes

9 Repository Map

Table 8: Repository layout (`github.com/Anamitra-Sarkar/yoga-posture-correction-system`).

Path	Contents
`frontend/src/` (`pages`, `hooks`, `utils`)	The web app: practice screen and landing page; the pipeline hook (`useYogaPipeline`); geometry, visibility, sequence buffer, CPU pose engine, offline coach
`frontend/android/`, `mobile/`	Capacitor Android project; Expo companion app
`backend/app/main.py`, `routers/`	FastAPI app and the `/api` routes
`backend/app/models/`	`mlp.py` (3-head ResMLP), `sequence.py` (ST-GCN)
`backend/app/services/`	`cascade.py` (decision logic), `hf_loader.py` (weights), `correction.py` (coaching), `occlusion.py`
`backend/app/utils/`	`geometry.py` (angles, window normalisation), `rules_classifier.py` (angle bands, rule engine)
`backend/tests/`	Unit and endpoint tests
`modal/`, `planning/`	Data preparation and training scripts, Kaggle kernels
`docs/`	Cascade, sequence-model and benchmark notes; this document
`paper/`, `report/`	Research paper and project report (LaTeX)
`.github/workflows/`	Space sync and Android build